数字逻辑基础与Verilog设计
---组合篇

第3章 算术运算电路

计算机中数的表示方法

- 分类
  - 按照有无符号
    - 无符号数
    - 有符号数
      - 原码
      - 反码
      - 补码
  - 按小数点是否固定
    - 定点数
      - 定点小数
      - 定点整数
    - 浮点数

数的表示方法在前期课程中均有涉及，本课程只讲解涉及的算术运算电路及其**Verilog**实现。

本章内容

- 无符号算术运算
  - 全加器，行波进位加法器，快速加法器
  - 应用实例：1位BCD码加法，乘法器；
  - 用Verilog设计算术运算电路
- 有符号算术运算

9. 无符号数的算术运算

- 无符号数：
  - 正数，n位无符号二进制整数为：$B=b_{n-1}b_{n-2}\cdots b_1b_0$
- 二进制全加器：$X= x_{n-1}x_{n-2}\cdots x_1x_0$, $Y= y_{n-1}y_{n-2}\cdots y_1y_0$

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

s_i奇数个1：异或

c_i \ x_iy_i	00	01	11	10
0		1		1
1	1		1	

$s_i = x_i \oplus y_i \oplus c_i$

c_i \ x_iy_i	00	01	11	10
0			1	
1		1	1	1

$c_{i+1}=x_iy_i+x_ic_i+y_ic_i$

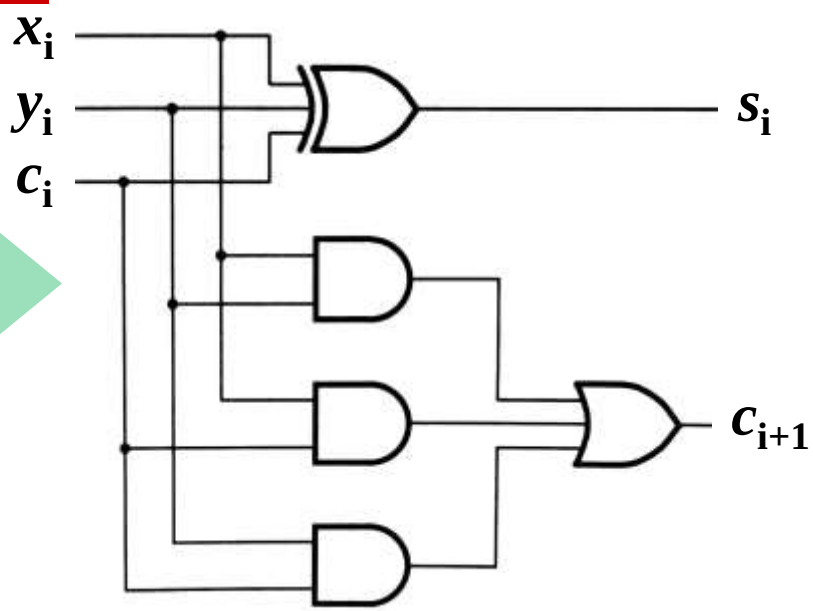
$c_{i+1}= x_iy_i+(x_i\oplus y_i)c_i$

9. 无符号数的算术运算

- 二进制全加器的实现
  - 使用“异或”门实现

$$s_i = x_i \oplus y_i \oplus c_i$$

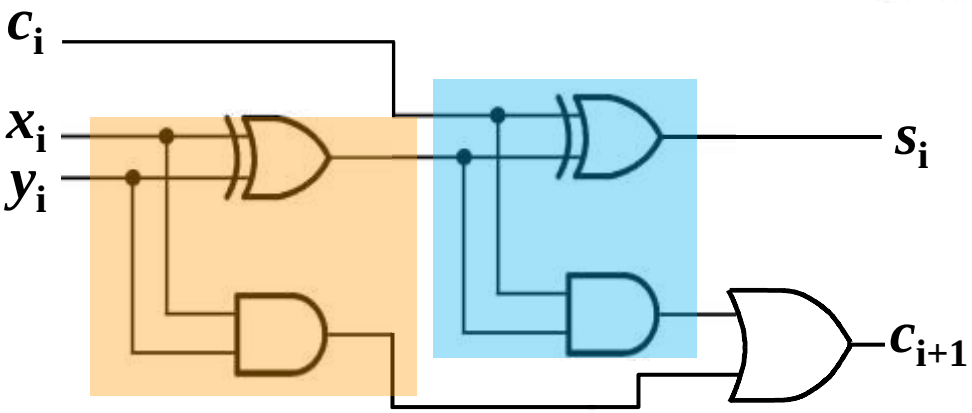
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$



  - 层次化设计方法：半加器

x_i	y_i	c_{i+1}	s_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

半加器真值表



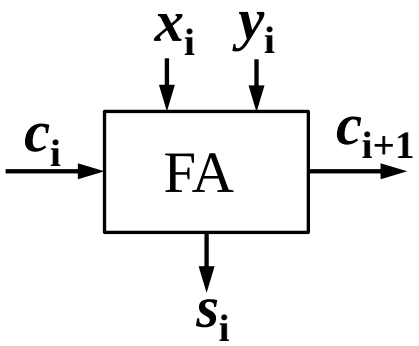


Figure 3.3-3.4. Implementation of the full-adder circuit

9. 无符号数的算术运算

- 二进制全加器的Verilog实现

```
module fulladd (
    input Cin, x, y ,
    output s, Cout
);
    xor (s, x, y, Cin);
    and (z1, x, y);
    and (z2, x, Cin);
    and (z3, y, Cin);
    or (Cout, z1, z2, z3);

endmodule
```

```
module fulladd (
    input Cin, x, y,
    output s, Cout
);
    assign s = x ^ y ^ Cin;
    assign Cout = (x & y)
                | (x & Cin)
                | (y & Cin);

endmodule
```

图 3.18， 3.20. Verilog code for the full-adder using gate level primitives and continuous assignment.

9. 无符号数的算术运算

- **行波进位加法器(ripple-carry adder)**
  - **n位二进制数相加用n个全加器: 将低位的进位输出c_i接到高位的进位输入c_{i+1}。**
  - **优点：结构简单；**
  - **缺点：运算速度慢，要经过n级FA的延时。**

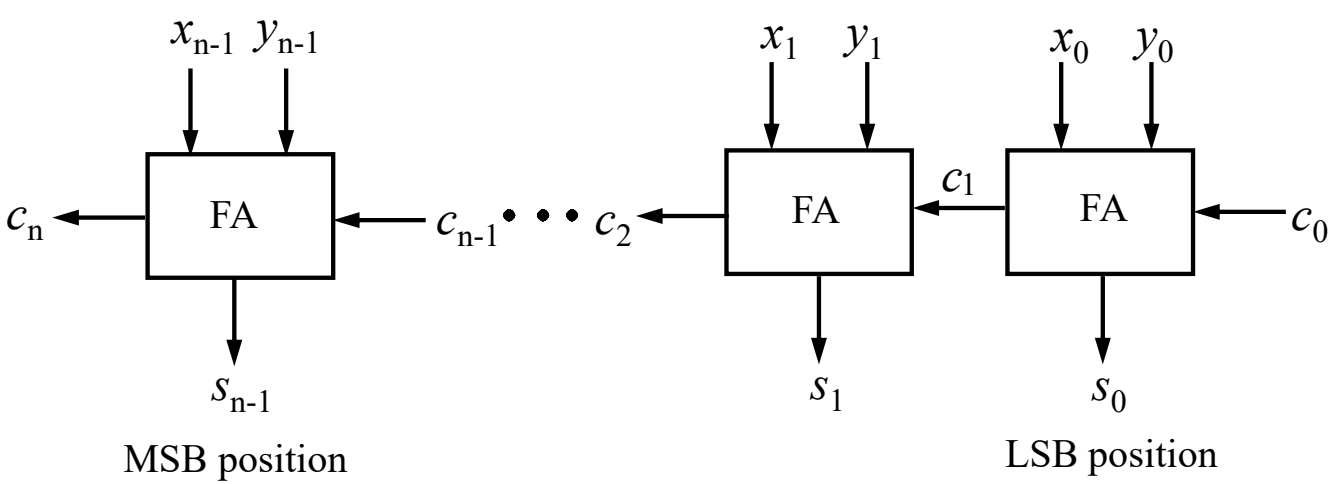


Figure 3.5. An *n*-bit ripple-carry adder.

9. 无符号数的算术运算

行波进位加法器的代码实现1：

```
module adder4 (
    input cin,
    input [3:0] x, y,
    output [3:0] s,
    output cout
);
    wire [3:1] c;
    fulladd stage0 (cin, x[0], y[0], s[0], c[1]);
    fulladd stage1 (c[1], x[1], y[1], s[1], c[2]);
    fulladd stage2 (c[2], x[2], y[2], s[2], c[3]);
    fulladd stage3 (c[3], x[3], y[3], s[3], cout);
endmodule
```

```
module fulladd (
    input Cin, x, y,
    output s, Cout
);
    assign s = x ^ y ^ Cin,
    assign Cout = (x & y)
                | (x & Cin)
                | (y & Cin);
endmodule
```

采用模块调用的方式实现更利于进行综合优化。

Figure 3.23. A four-bit adder using vectors.

9. 无符号数的算术运算

行波进位加法器的代码实现2：

```
module addern  #
(
    parameter n=32
)
(
    input cin,
    input [n-1:0] x, y,
    output reg [n-1:0] s,
    output reg cout
);
    reg [n:0] c;
    integer k;
```

```
    always @(x, y, cin)
    begin
        c[0] = cin;
        for (k = 0; k < n; k = k+1)
        begin
          s[k] = x[k] ^ y[k] ^ c[k];
          c[k+1] = (x[k]&y[k])
                 | (x[k]&c[k])
                 | (y[k]&c[k]);
        end
        cout = c[n];
    end

endmodule
```

端口宽度参数化

采用always过程语句建模

Figure 3.24. A generic specification of a ripple-carry adder.

9. 无符号数的算术运算

行波进位加法器的代码实现3：

```
module adder_n (Cin, X, Y, S, Cout);
    parameter n=32;
    input Cin;
    input [n-1:0] X, Y;
    output [n-1:0] S;
    output Cout;
    wire [n:0] C;
    genvar i;   //generate中的循环变量类型
    assign C[0]=Cin;
    assign Cout = C[n];
    generate  //允许在for循环中包含实例化语句
      for(i=0;i<=n-1;i=i+1)
        begin:addbit //实例化名字addbit[i].stage
          fulladd stage(C[i], X[i], Y[i], S[i], C[i+1]);
        end
    endgenerate
endmodule
```

```
module fulladd (
   input Cin, x, y,
   output s, Cout
);
   assign s = x ^ y ^ Cin,
   assign Cout = (x&y)
                 |(x&Cin)
                 |(y&Cin);
endmodule
```

利用generate实现循环实例化.

切记：always过程化语句中不可以模块调用。

9. 无符号数的算术运算

- 超前进位加法器(Carry Look - ahead adder)

•为了提高速度，若使进位信号不逐级传递，而是运算开始时，即可得到各位的进位信号，采用这个原理构成的加法器，就是超前进位（Carry Look－ahead）加法器，也称为快速进位（Fast carry）加法器。

•第 i 位进位信号c_{i+1}：

$$c_{i+1}=x_i y_i+x_i c_i+y_i c_i$$

$$c_{i+1}=x_i y_i + (x_i+y_i)c_i$$

令：$g_i = x_i y_i$ (生成函数)， $p_i = x_i + y_i$ (传播函数)

$$c_{i+1}= g_i + p_i c_i$$

9. 无符号数的算术运算

- 超前进位加法器(Carry Look - ahead adder)

$$c_{i+1} = g_i + p_i c_i$$
$$= g_i + p_i (g_{i-1} + p_{i-1} c_{i-1})$$
$$= g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1}$$
$$= g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots p_i p_{i-1} \cdots p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$

$$g_i = x_i y_i ,$$
$$p_i = x_i + y_i$$

$$s_i = x_i \oplus y_i \oplus c_i$$

- 由于p_i与g_i均可快速获取，在位数不多的情况下，提高了运算速度。但电路的复杂性，随着位数n的增加迅速增大。
- 可采取层次化设计方法来减低复杂度，例：设计32位加法器，分为4个8位加法器模块，每个模块采取超前进位，模块间采用行波进位或超前进位。

9. 无符号数的算术运算

- 应用实例1：设计一个电路实现一个8位无符号数乘以3.

分析：8位无符号数的3倍需要用10位二进制表示。

设 $A = a_7a_6\cdots a_1a_0$，乘积 $P = p_9p_8\cdots p_1p_0$。

- 两种设计思路：
  - 3A=(A+A)+A，两个加法器级联，一个8位，一个9位。
  - 3A=2A+A，将A左移一位可以得到2A= $a_7a_6\cdots a_1a_0 0$，采用一个9位加法器实现。

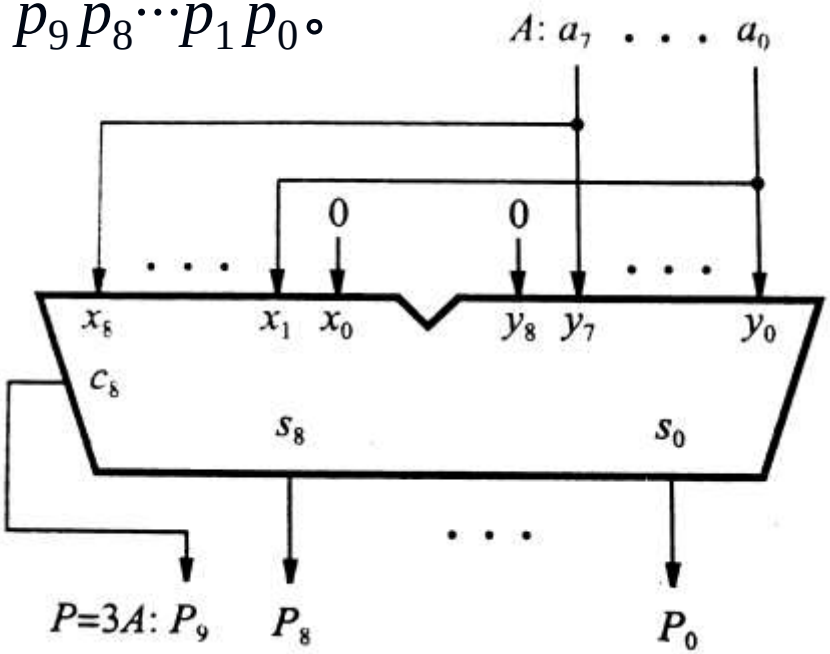
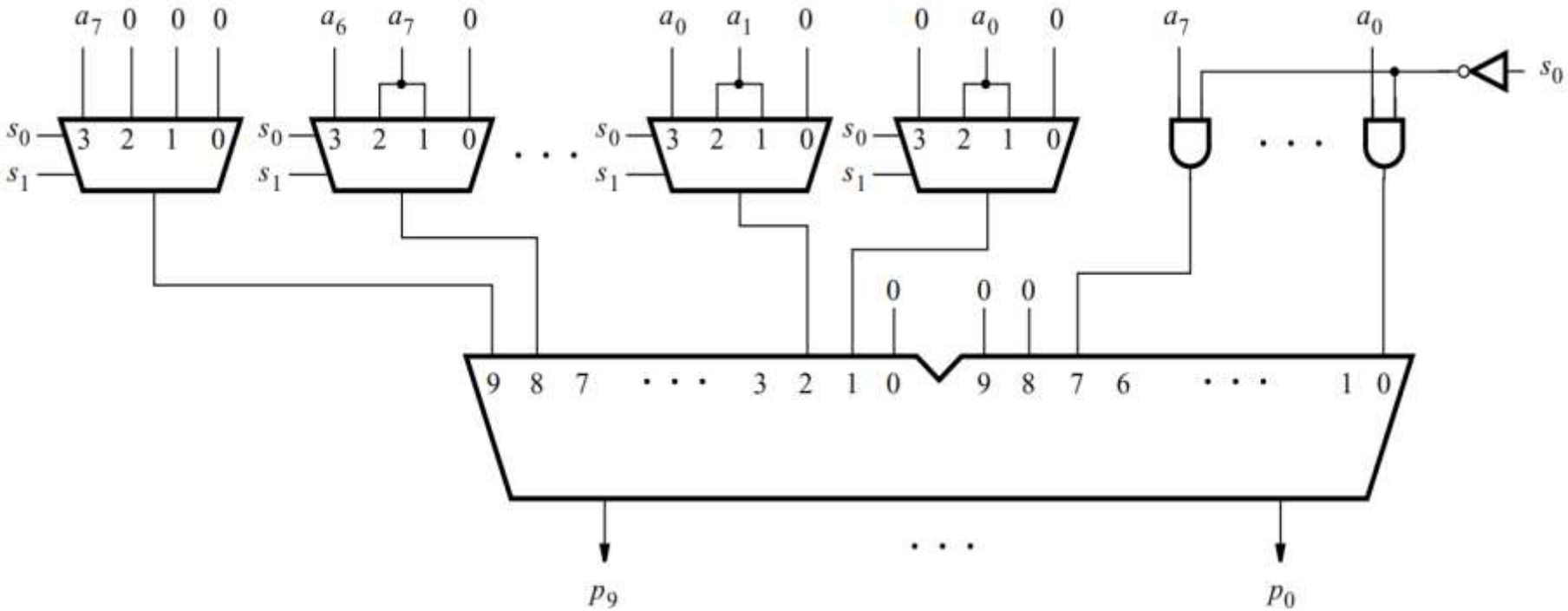


Figure 3.6. An eight-bit unsigned number by 3.

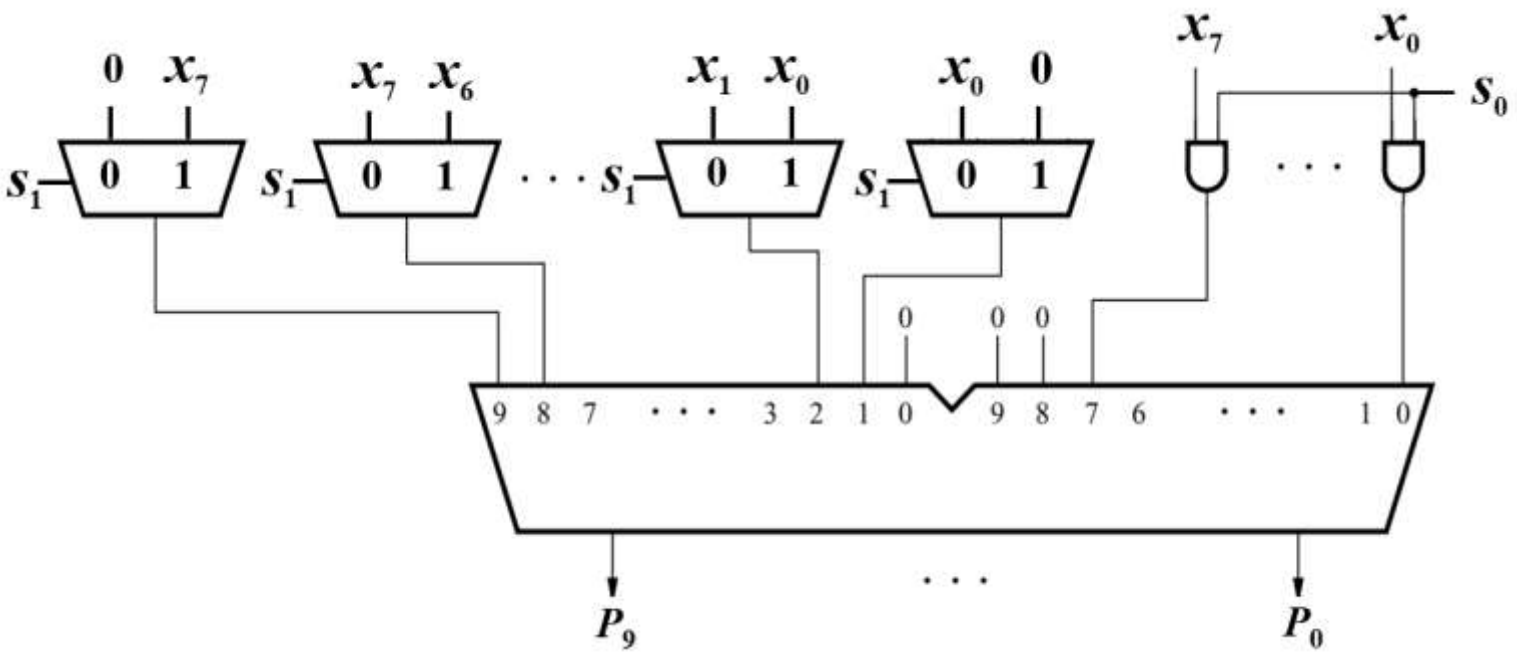
9. 无符号数的算术运算

- 应用实例2：(习题4.30)
- 设计一个电路，实现一个8位无符号数乘以1，2，3，4.
- 思路：采用4-1多路选择器，S_1S_0分别为00，01，10，11实现乘1，2，3，4功能。



9. 无符号数的算术运算

- 应用实例2：
- 设计一个电路，实现一个8位无符号数乘以2，3，4，5.
- 思路：采用2-1多路选择器，S_1S_0分别为00，01，10，11实现乘2，3，4，5功能。



9. 无符号数的算术运算

- 应用实例3： 1位BCD码的加法电路（P92，3.7.2）
- BCD码：二进制编码十进制数表示法；
- 约定用X、Y表示两个BCD数：
  - ⑩$X= x_3x_2x_1x_0$, $Y= y_3y_2y_1y_0$；
  - ⑩用S表示和：$S= s_3s_2s_1s_0$ ， c_{out}表示进位；
- 将BCD码看作为二进制数进行加法运算，令$Z= z_4z_3z_2z_1z_0$，$Z = (X)_2+ (Y)_2$

9. 无符号数的算术运算

当 Z <= 9:

BCD加法等效于两个四位二进制的加法；
$S=Z$，
进位 $c_{out}=0$；

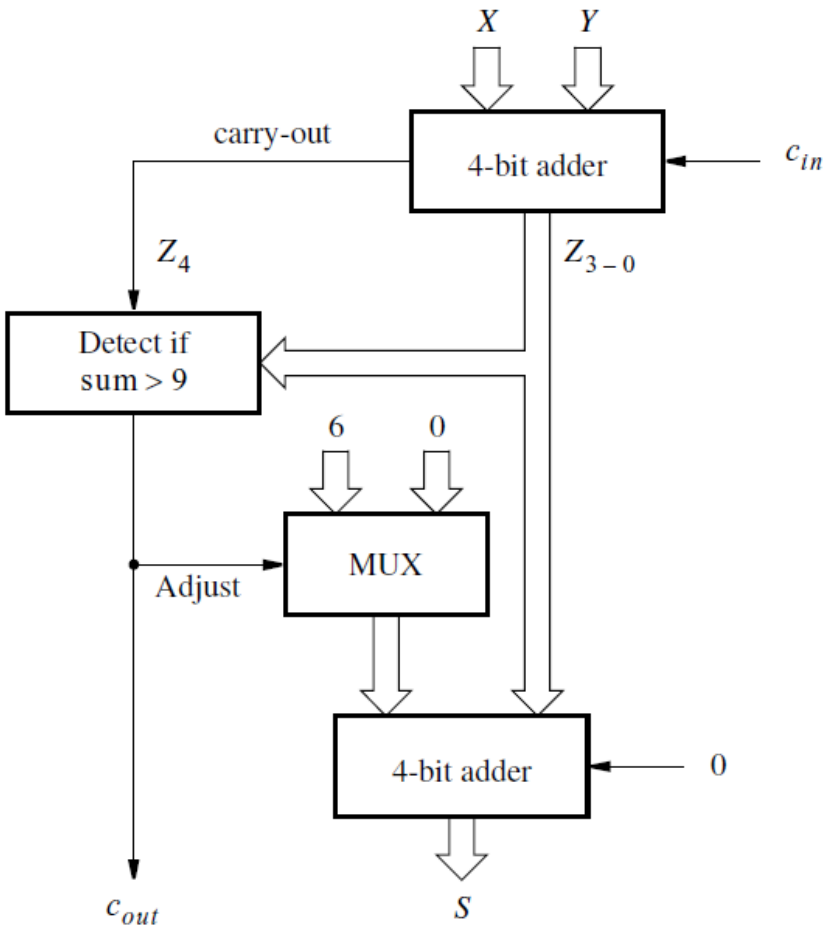
当 Z > 9:

“和”位产生错误，需要修正: $S=Z+6$，
进位 $c_{out}=1$。

十进制数	二进制数相加的“和数”					BCD 码十进制数相加“和数”				
	Z_4	Z_3	Z_2	Z_1	Z_0	$Cout$	S_3	S_2	S_1	S_0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

9. 无符号数的算术运算

- **1位BCD码的加法电路**




```
module bcdadd (
    input cin,
    input [3:0] x, y ,
    output reg [3:0] s ,
    output reg cout
);
    reg [4:0] z;
    always@ (x, y, cin)
    begin
        z = x + y + cin;
        if (z < 10)
            {cout, s} =  z;
        else
            {cout, s} =  z + 6;
    end
endmodule
```

Figure 3.39-3.40. Block diagram and Verilog code for a one-digit BCD adder.

9. 无符号数的算术运算

- **1位BCD码的加法电路**
- 产生进位c_{out}=1条件

$$c_{out} = z_4 + \sum m(10,11,12,13,14,15)$$

$$c_{out} = z_4 + z_3z_2 + z_3z_1$$

z_1z_0 \ z_3z_2	00	01	11	10
00			1	
01			1	
11			1	1
10			1	1

z_3z_2

z_3z_1

- 和位的产生

$$\begin{array}{rcccc} & z_3 & z_2 & z_1 & z_0 \\ + & 0_c & c_{out} & c_{out} & 0 \\ \hline & s_3 & s_2 & s_1 & s_0 \end{array}$$

$s_0 = z_0$

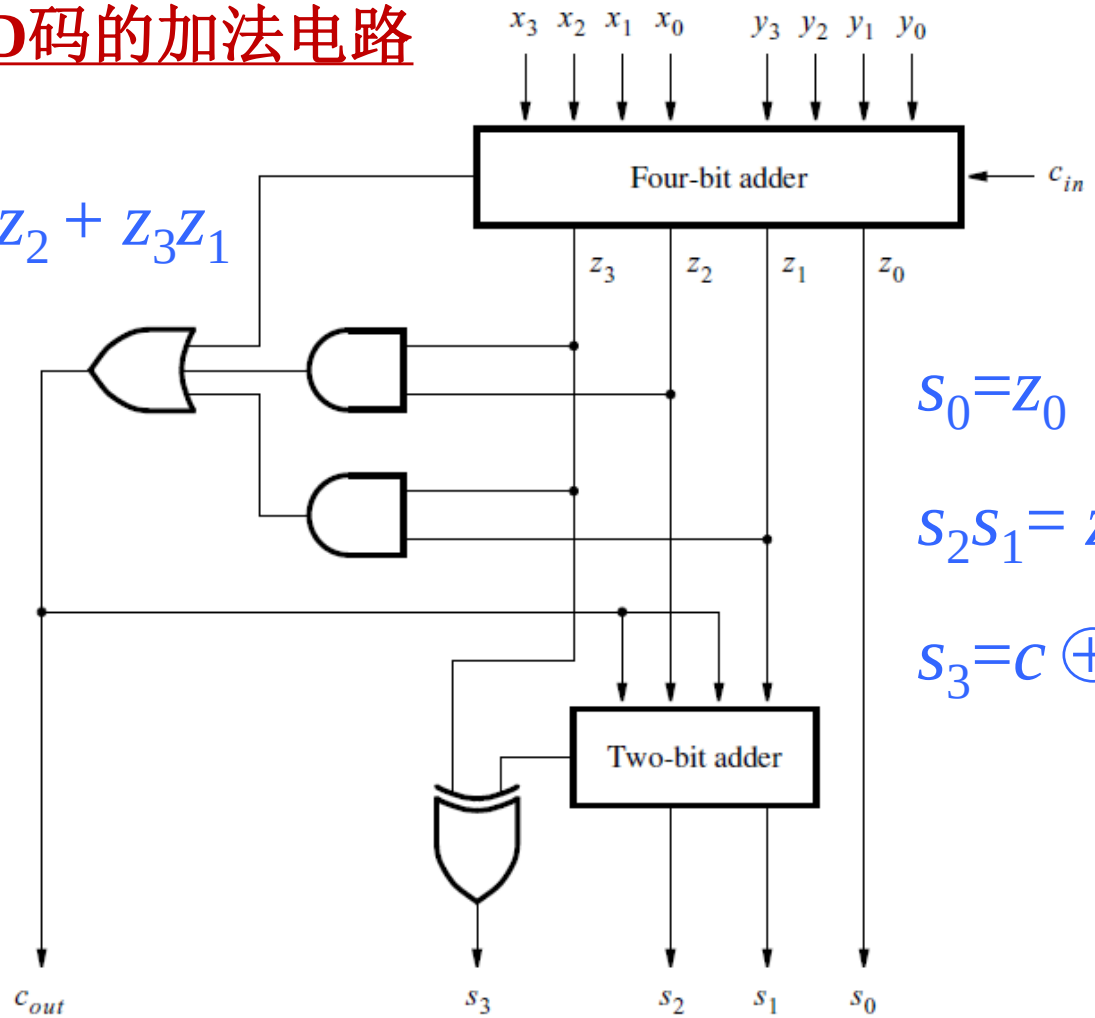
$s_2s_1 = z_2 z_1 + c_{out}\, c_{out}$（算术加）

$s_3 = c \oplus z_3$

9. 无符号数的算术运算

- **1位BCD码的加法电路**

$c_{out} = z_4 + z_3z_2 + z_3z_1$



$s_0 = z_0$

$s_2s_1 = z_2 z_1 + c_{out}\, c_{out}$

$s_3 = c \oplus z_3$

Figure 3.41. Circuit for a one-digit BCD adder.

9. 无符号数的算术运算

- 应用实例4： n位无符号数的乘法
- 四位无符号数：$M= m_3m_2m_1m_0$, $Q= q_3\,q_2\,q_1\,q_0$;
- 例：四位无符号数的乘法

被乘数 M=14
乘数 Q =11

				1	1	1	0	
			×	1	0	1	1	
				1	1	1	0	
			1	1	1	0		
		0	0	0	0			
	1	1	1	0				
积 P =154	1	0	0	1	1	0	1	0

被乘数 M=14
乘数 Q =11

				1	1	1	0	
			×	1	0	1	1	
部分积0				1	1	1	0	
			+ 1	1	1	0		
部分积1			1	0	1	0	1	
		+ 0	0	0	0			
部分积2		0	1	0	1	0		
	+ 1	1	1	0				
积P	1	0	0	1	1	0	1	0

Figure 3.34. (a) Multiplication by hand; (b) Using multiple adders

9. 无符号数的算术运算

					m_3	m_2	m_1	m_0
					$\times\ q_3$	q_2	q_1	q_0
部分积0					m_3q_0	m_2q_0	m_1q_0	m_0q_0
				$+\ m_3q_1$	m_2q_1	m_1q_1	m_0q_1	
部分积1			$PP1_5$	$PP1_4$	$PP1_3$	$PP1_2$	$PP1_1$	
		$+\ m_3q_2$	m_2q_2	m_1q_2	m_0q_2			
部分积2	$PP2_6$	$PP2_5$	$PP2_4$	$PP2_3$	$PP2_2$			
	$+\ m_3q_3$	m_2q_3	m_1q_3	m_0q_3				
积P	P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0

Figure 3.34. Multiplication of unsigned numbers

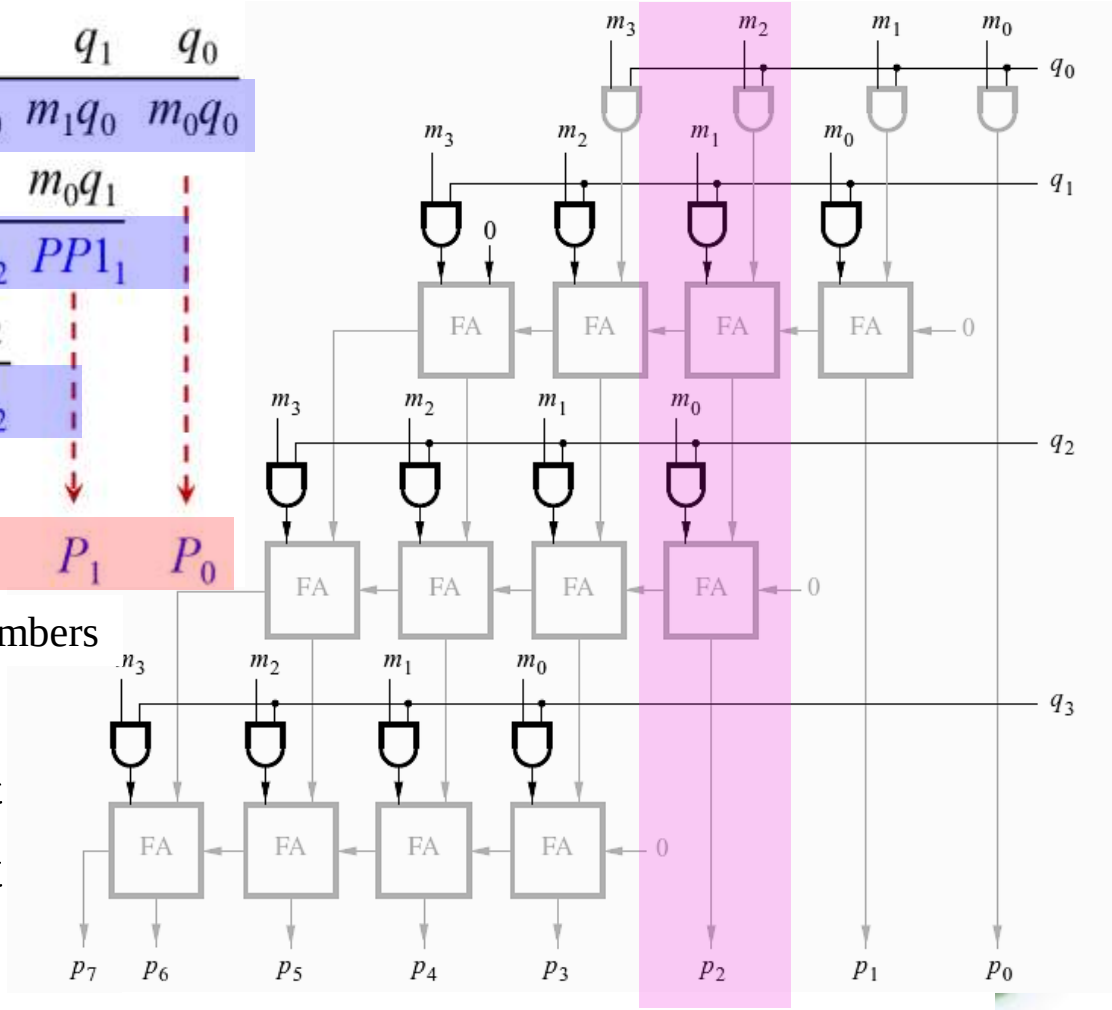


Figure 3.35. A 4x4 multiplier circuit.

- 每行四个全加器，构成行波进位加法器，进位信号的传递造成的延时降低了电路速度。

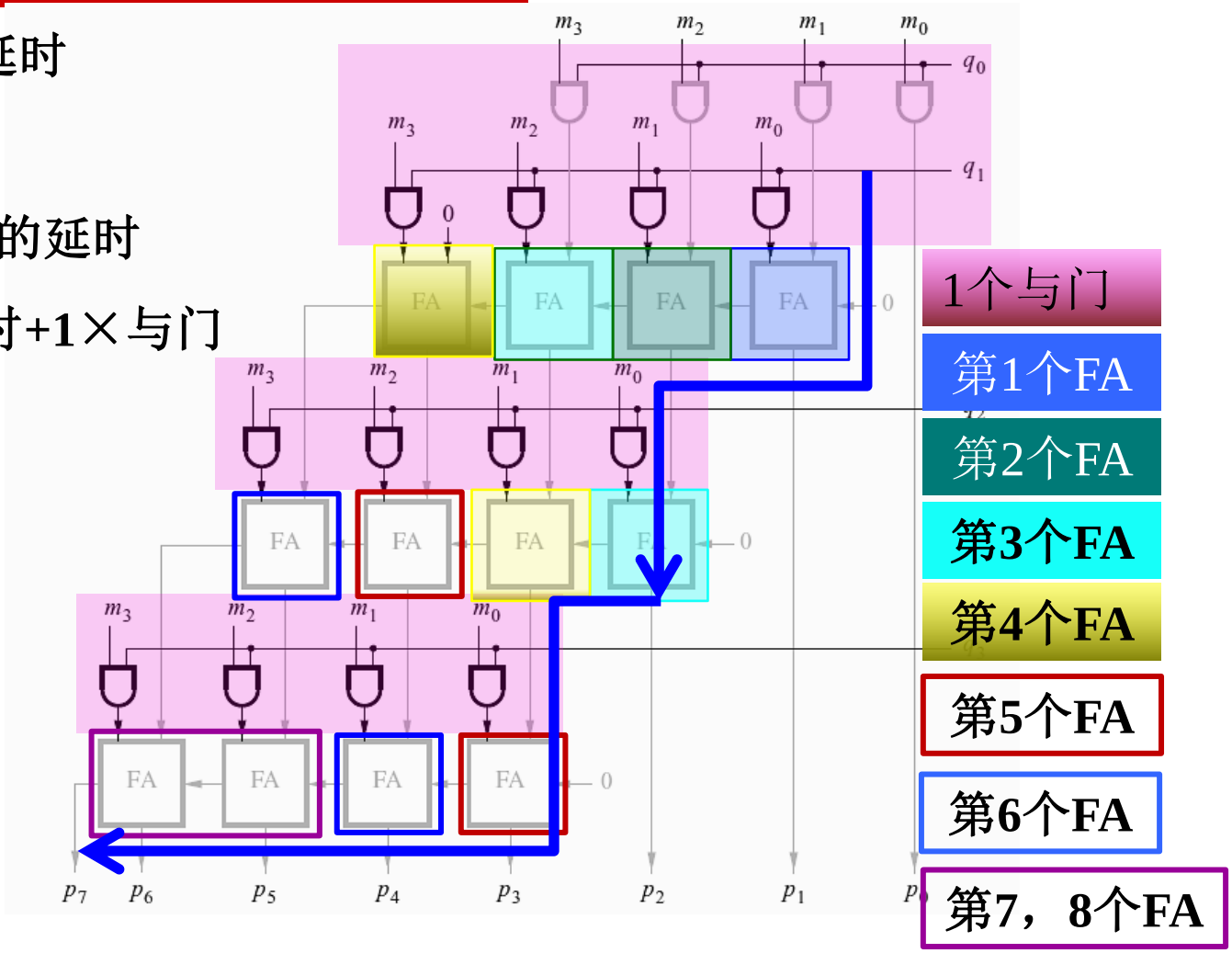
9. 无符号数的算术运算

分析电路的关键延时

- 关键延时：

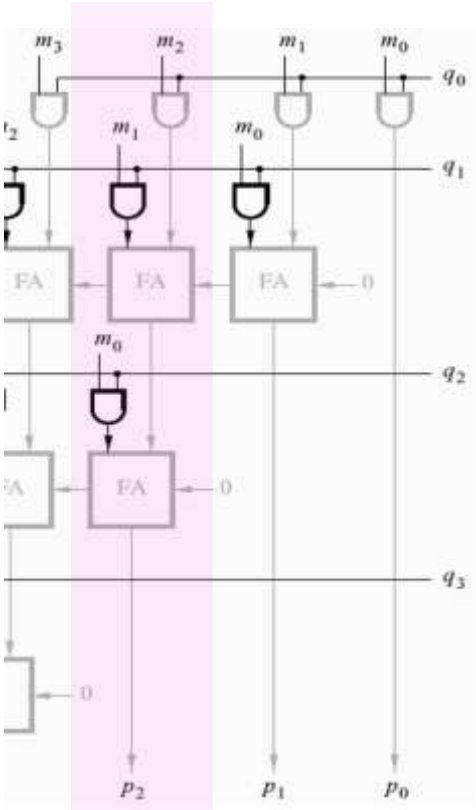
  最长路径带来的延时

- 共：8×FA延时+1×与门



9. 无符号数的算术运算

- 为了提高速度，交换每一位输出值p_i的“加数”顺序，使得每一行四个加法器并行操作；
- 相当于是将进位信号“保存”下来，加到下一行的正确位置；但最后一行，仍然要用行波进位加法器。关键延时：6×FA+1×与门



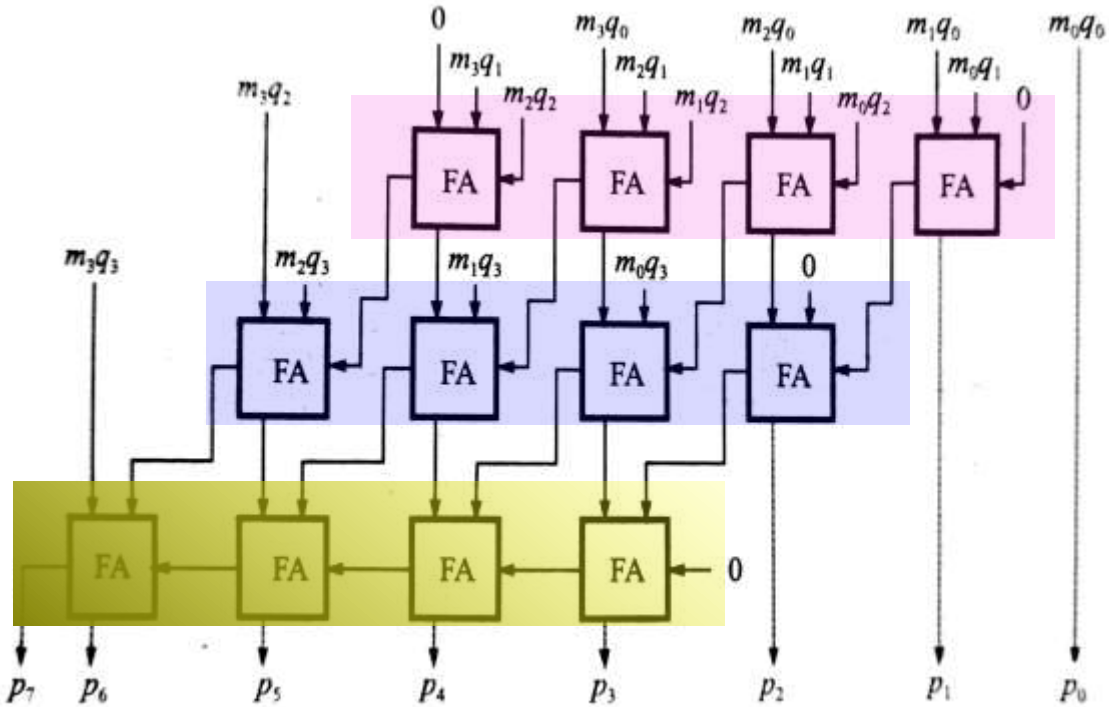


图 3.48 进位保留加法器阵列

本章作业

- 验证3.8，3.9
- 3.14，3.16，3.17,
- 3.18，3.19，3.23,3.25(选做)